LLM for Software Engineering Course Projects

2025-2026

Project Assignment

- Teams of **5 people**
- Select 3 project proposals (at least one T, one A – see later) that you would like to do
- We will assign you – if possible – **one** of the projects that you have chosen

- Team management link (be careful when modifying it!!!) : https://docs.google.com/spreadsheets/d/1mswRyffUWuquIXr7dreP5cwSWOM4KOuyURbuvzoozms/edit?gid=0#gid=0

- Deadline for team and proposal selection: **November 30**
- Assignment of projects and project start: **December 1**

Project Evaluation

- Deadline for project hand-in: before the beginning of next academic year (September, 2025)

- Deadlines to have the LLM exam registered in a specific session:
  - Winter session -> January 31, 2026
  - Summer session -> June 30, 2026
  - Autumn session -> September 15, 2026

- Project points: 15/30 of the exam

- You will have to discuss the project (30 minutes presentation over slides, including Q&A)

Project Delivery

- You will receive a document with a template to fill with your methodology and result **and specific instructions for your team**

- To deliver the project, you have to submit:
  - The document describing your project work, a technical report created with Overleaf
  - The link to a GitHub project with your replication package

Project Categories

- **T (SemEval Task):** technical projects in which you will apply LLM models to solve SemEval challenges.
  - Responsible: prof. Flavio Giobergia

- **A (Application):** projects in which you will analyze the effectiveness of the application of LLMs in various Software Engineering tasks.
  - ResponsiblE: prof. Riccardo Coppola

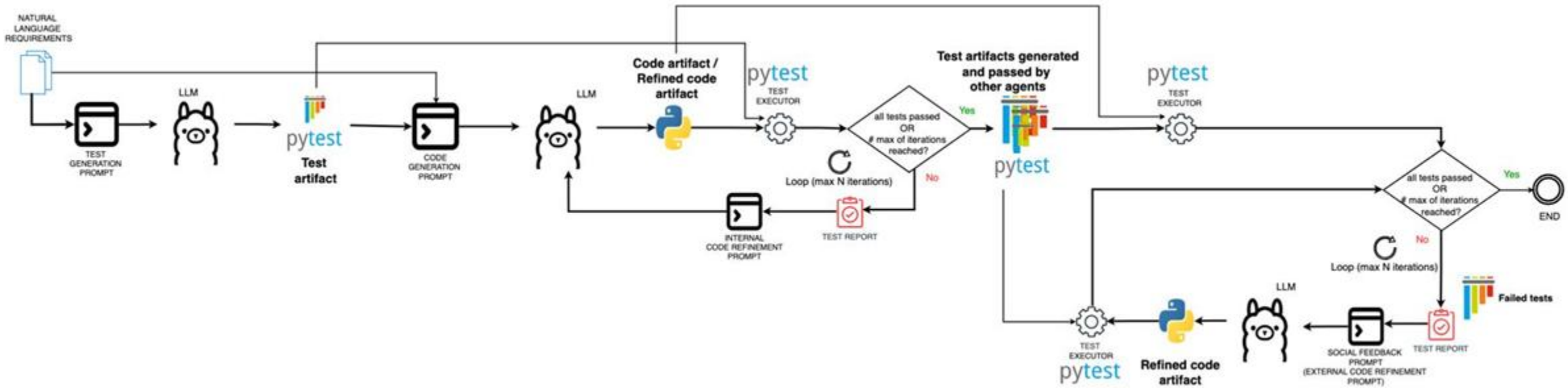
SemEval Tasks

- **https://semeval.github.io/SemEval2026/tasks**

A1: LLM Agents for Collaborative Test Case Generations

- Software testing often requires collaboration between testers, developers, domain experts, and tools. Traditional automated test generation tools cannot emulate multi-perspective reasoning (e.g., user intention, edge case exploration, domain knowledge). Recent advances in multi-agent LLM architectures allow for collaborative workflows where agents work differently to generate the same artefacts.

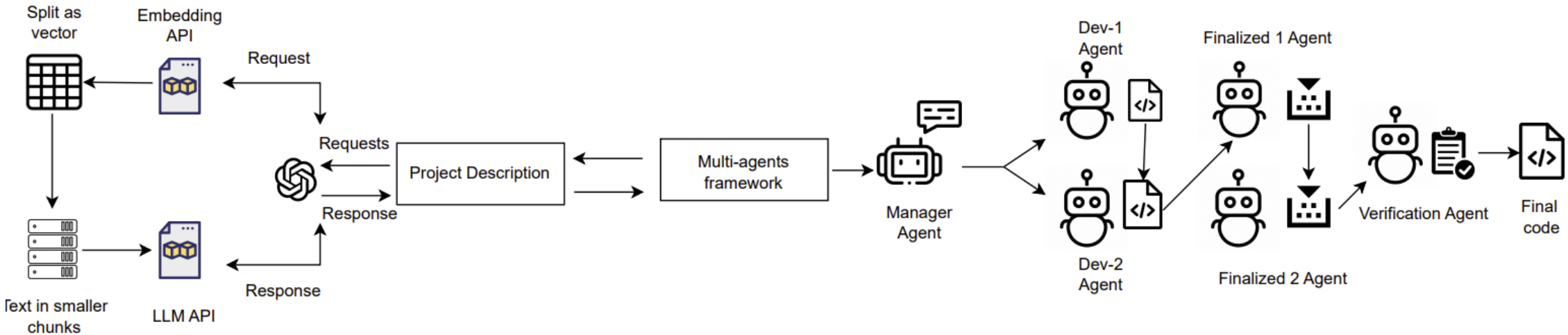
- Research Questions
  - How effective are LLM agents in generating comprehensive and diverse test cases?
  - Does agent collaboration outperform single-model test generation?
  - What patterns of collaboration lead to higher-quality test cases?


NATURAL LANGUAGE REQUIREMENTS
TEST GENERATION PROMPT
LLM
pytest
Test artifact
CODE GENERATION PROMPT
LLM
Code artifact / Refined code artifact
pytest
TEST EXECUTOR
all tests passed OR # max of iterations reached?
Yes
No
Loop (max N iterations)
TEST REPORT
INTERNAL CODE REFINEMENT PROMPT
Test artifacts generated and passed by other agents
pytest
pytest
TEST EXECUTOR
all tests passed OR # max of iterations reached?
Yes
END
No
Loop (max N iterations)
Failed tests
TEST REPORT
SOCIAL FEEDBACK PROMPT (EXTERNAL CODE REFINEMENT PROMPT)
LLM
Refined code artifact
TEST EXECUTOR
pytest

A2: Architectures for Code Development with LLMs

- LLMs can generate code, but single-prompt interactions often fail on long or complex development tasks. Multi-agent architectures may improve quality by splitting responsibilities (design, planning, writing, reviewing, debugging).

- Research Questions
  - Which architectures produce higher-quality and more maintainable code?
  - How do agent coordination strategies impact correctness?
  - Does modular role separation improve ode generation?


Split as vector
Embedding API
Request
Requests
Response
Project Description
Multi-agents framework
Manager Agent
Dev-1 Agent
Dev-2 Agent
Finalized 1 Agent
Finalized 2 Agent
Verification Agent
Final code
Text in smaller chunks
LLM API
Response

A3: Generating Software Requirements Through Abstractions

- Requirements are often ambiguous, inconsistent, and non-standardized. LLMs can support requirement authoring, but without clear abstraction layers (goal → feature → scenario → constraint), generated requirements remain too “flat”.

- Research Questions
  - Can LLMs reliably generate requirements at distinct abstraction levels?
  - Does hierarchical generation improve clarity and completeness?
  - Can LLMs detect inconsistencies between levels?

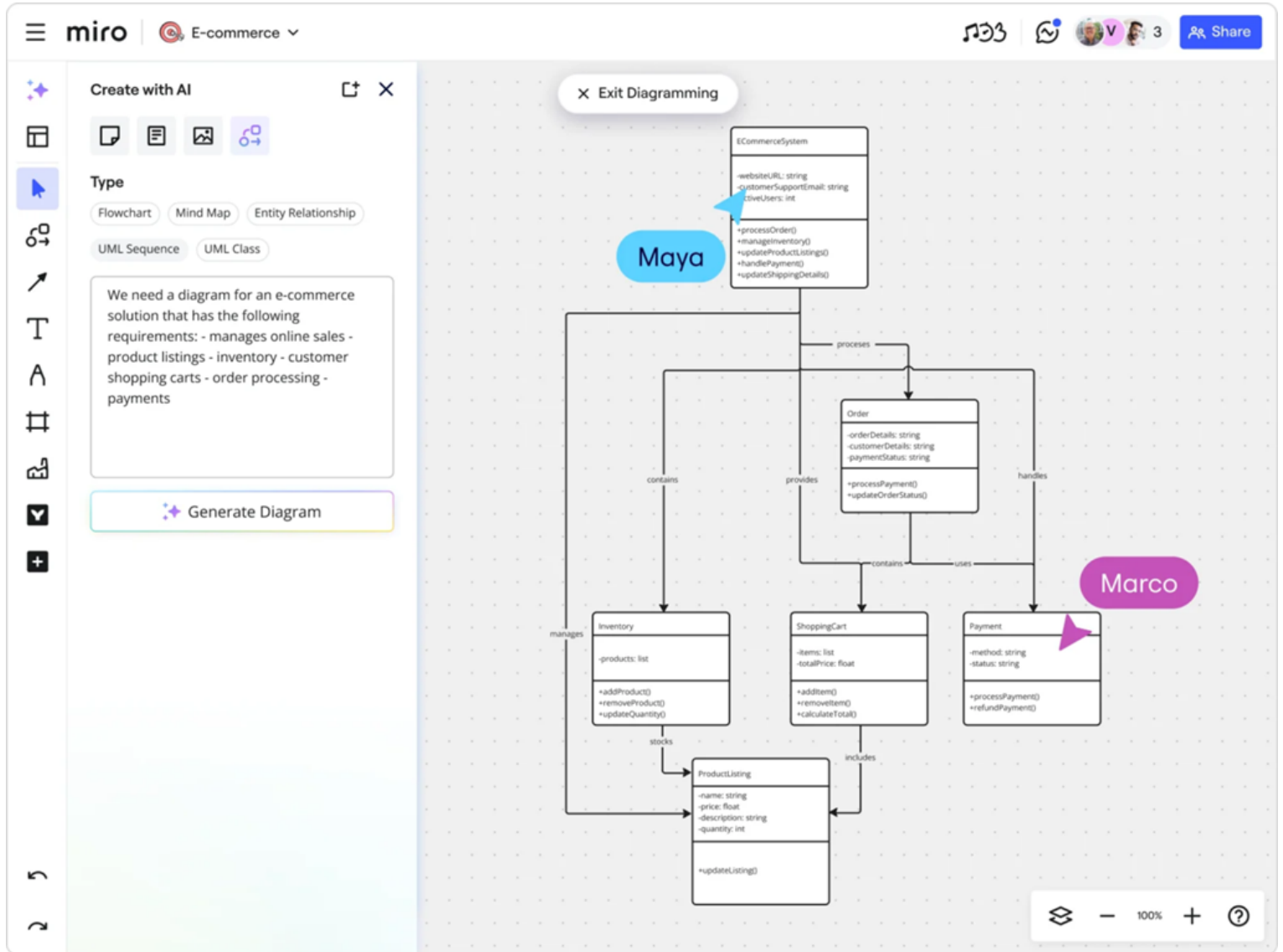
The product shall be available during normal business hours . As long as the user has access to the client PC

the system will be available 99 % of the time during the first six months of operation

A4: Generating and Correcting Design Diagrams with LLMs

- Design diagrams (UML class diagrams, sequence diagrams, state machines) are essential for software engineering but often missing or outdated. LLMs can help generate diagrams from text or correct existing diagrams.

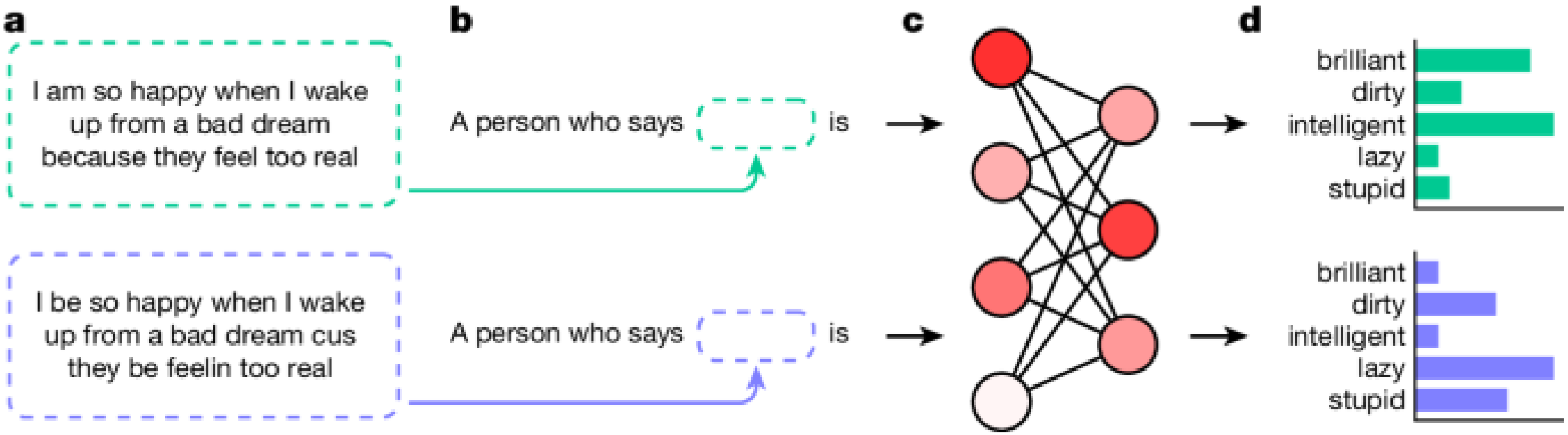
- Research Questions
  - How accurate are LLMs at generating formal design diagrams from natural language?
  - Can multi-agent pipelines improve diagram consistency?
  - How effective are LLMs at detecting and correcting structural errors in diagrams?


miro
E-commerce
3
Share
Create with AI
Type
Flowchart
Mind Map
Entity Relationship
UML Sequence
UML Class
We need a diagram for an e-commerce solution that has the following requirements: - manages online sales - product listings - inventory - customer shopping carts - order processing - payments
Generate Diagram
Exit Diagramming
Maya
Marco
ECommerceSystem
-websiteURL: string
-customerSupportEmail: string
-activeUsers: int
+processOrder()
+manageInventory()
+updateProductListings()
+handlePayment()
+updateShippingDetails()
proceses
Order
-orderDetails: string
-customerDetails: string
-paymentStatus: string
+processPayment()
+updateOrderStatus()
contains
provides
handles
contains
uses
manages
Inventory
-products: list
+addProduct()
+removeProduct()
+updateQuantity()
ShoppingCart
-items: list
-totalPrice: float
+addItem()
+removeItem()
+calculateTotal()
Payment
-method: string
-status: string
+processPayment()
+refundPayment()
stocks
includes
ProductListing
-name: string
-price: float
-description: string
-quantity: int
+updateListing()
100%

A5: Analysis of Linguistic Stereotypes in Generative AI

- Generative models (LLMs, text-to-image systems) often reproduce cultural or linguistic stereotypes. Detecting such bias in generated outputs requires systematic linguistic analysis and structured evaluation.

- Research Questions
  - What types of linguistic stereotypes do LLMs reproduce?
  - Does prompt structure (zero-shot, role prompting, chain-of-thought) amplify or reduce bias?
  - Can multi-agent critique frameworks reduce stereotypical outputs?


a
I am so happy when I wake up from a bad dream because they feel too real
I be so happy when I wake up from a bad dream cus they be feelin too real
b
A person who says
is
A person who says
is
c
d
brilliant
dirty
intelligent
lazy
stupid
brilliant
dirty
intelligent
lazy
stupid